

هل سيحل الذكاء الاصطناعي في البرمجة محل جميع المبرمجين؟ نظرة إلى الواقع في القطاع

Zhihu • 2026-09-19

منذ أن تحسّنت نتائج البرمجة بالذكاء الاصطناعي تحسّناً كبيراً في نهاية 2025 ومطلع 2026، صار الذكاء الاصطناعي يحل محل جزء كبير من عمل المبرمجين في كتابة الشيفرة، بل معظمه أحياناً. وقد أثارت هذه الظاهرة بين المبرمجين مشاعر الهلع واليأس. وفي الوقت نفسه، استغلت شركات الذكاء الاصطناعي التقدم المحقق في هذا المجال لتضخّم وظائف الذكاء الاصطناعي بلا حدود، وتردد أن جميع ذوي الياقات البيضاء سيفقدون وظائفهم في غضون سنة أو سنتين. وسارعت وسائل الإعلام الذاتية كلها إلى اللحاق بالركب، تنشر الهلع وتصنع القلق.

وفي هذه المسألة، يلزمنا أن نفحص الأمر بجدية. فهل تستطيع البرمجة بالذكاء الاصطناعي أن تحل محل جميع المبرمجين؟ وإن لم تستطع، فما نسبة الاستبدال الفعلية؟

في الواقع، قدّمت في جواب كتبه قبل أشهر قليلة («أول سيف للذكاء الاصطناعي يقع على المبرمجين، فهل سيفقد المبرمجون وظائفهم بأعداد كبيرة؟») تقديري الخاص: لا يستطيع الذكاء الاصطناعي أن يحل محل العمل البشري كله، ونسبة الاستبدال من طرف إلى طرف لا تتجاوز 30%.

حين تدخل منظومات الآلات الذكية التي يكون الذكاء الاصطناعي قلبها الإنتاج على نطاق واسع، سيختفي عمل كتابة الشيفرة يدوياً أساساً. وبالطبع قد يظل تطوير الشيفرة الجوهرية المعقدة بنيوياً أو ذات متطلبات الأداء العالية جدّاً بحاجة إلى تدخل بشري مكثف، أما كتابة الشيفرة العادية الواسعة فسيحل الذكاء الاصطناعي محلها بالكامل. فكم شخصاً سيستبدل تحديداً؟ بحسب النتائج الفعلية لاستخدامي، ووفق ما ردّ به آخرون على Zhihu، يتراوح رفع الكفاءة بين 20% و50% تقريباً. وإذا استقرّت البرمجة بالذكاء الاصطناعي فعلياً (وهي الآن في مرحلة تجربة واسعة في القطاع)، وبحسب تقدير نسبة المطوّرين الصرفين بـ60%، فسيقلّص ذلك نحو 12%-30% من وظائف المبرمجين.

لقد مضى نصف عام على الأقل على استقرار البرمجة بالذكاء الاصطناعي على نطاق واسع، ولا سيما في الصين والولايات المتحدة، حيث تُلزم الشركات الكبرى موظفيها تقريباً باستخدام وكلاء الذكاء الاصطناعي في التطوير. فما النتيجة فعلاً؟

بحسب تقرير ماكنزي^[1]، وفيما يخص المبرمجين «القادرين على استخدام الذكاء الاصطناعي»، يمكن رفع الإنتاجية بنحو 16%-30%، وهو ما يوافق تقديري السابق في العموم.

Figure 1 

وفي دراسة حديثة^[2]: تبين أيضاً أنه رغم أن البرمجة بالذكاء الاصطناعي ترفع سرعة توليد الشيفرة رفعاً كبيراً، فإن أثر ذلك عند الوصول إلى الإصدار النهائي للمنتج لا يتجاوز رفع الكفاءة بنحو 30%. وهذا يتوافق مرة أخرى مع دراسة ماكنزي المذكورة أعلاه ومع تقديري السابق.

أدوات الذكاء الاصطناعي التوليدي المستخدمة في تطوير البرمجيات تتقدم بسرعة مذهلة؛ فقد انتقلت في سنوات قليلة من الإكمال التلقائي البسيط للشفيرة إلى وكلاء برمجة أذكاء مستقلين تمامًا (coding agents). فهل تترجم هذه التعزيزات في القدرات إلى نمو في الإنتاجية على مستوى المهام في العالم الواقعي؟ وحتى لو ترجمت، فهل يعني تحسّن الكفاءة على مستوى المهام زيادة في الناتج النهائي؟ هذه الأسئلة جوهرية لفهم الأثر الاقتصادي للذكاء الاصطناعي. ومن خلال دراسة أحداث مقترنة بالمطورين، نجد أن الأثر في رفع الإنتاجية على مستوى المهام يزداد وضوحًا مع تعاقب أجيال أدوات البرمجة بالذكاء الاصطناعي: فقد زادت خاصية الإكمال التلقائي حجم الشيفرة المُودّعة أسبوعيًا بنحو 30%، ورفع الوكلاء المتزامنون (sync agents) هذا الأثر التراكمي إلى نحو 180%، ثم رفعه الوكلاء غير المتزامنين (async agents) إلى نحو 240%. غير أن هذه المكاسب تتضاءل بحدّة حين نصعد في المستويات إلى مرحلة الإنتاج: فرغم أن الوكلاء المتزامنين زادوا عدد أسطر الشيفرة بنسبة 958% وطلبات السحب (pull requests) بنسبة 86%، فإن عدد الإصدارات المنشورة في النهاية لم يزد إلا بنسبة 20%. ويظهر نشاط البرمجة الإجمالي على GitHub القاعدة نفسها.

Figure 2 

المصدر: <https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

وليس البحث وحده هو الذي يؤكد محدودية البرمجة بالذكاء الاصطناعي، بل إن المطورين والمديرين الذين يستخدمون وكلاء الذكاء الاصطناعي في عملهم يلمسون ذلك لمسًا مباشرًا. فقد استفاق المبرمجون من صدمتهم الأولى تجاه البرمجة بالذكاء الاصطناعي، وأخذوا يدركون محدوديتها عبر التطبيق الإنتاجي الفعلي. فمثلاً، كتب Cal Newport (أستاذ علوم الحاسوب في جامعة جورجتاون) مقالًا في مدونته^[3] يتناول فيه تحوّل أحد المبرمجين المخضرمين في وادي السيليكون خلال الأشهر الستة الماضية؛ يحكي فيه عن مطوّر ارتفعت إنتاجيته عدة أضعاف بعد البرمجة بـ Claude Code فصاح «really, really good»، ثم بعد حادثتين في بيئة الإنتاج سببتهما شيفرة الذكاء الاصطناعي، كاد يفقد وظيفته على إثرهما، فخابت آماله في البرمجة بالذكاء الاصطناعي، وعاد إلى كتابة الشيفرة الجوهريّة بيده، وقصر استخدام الذكاء الاصطناعي على كتابة حالات الاختبار والسكريبتات وما شابهها. ويرى هذا المطور أن البرمجة على الطريقة القديمة بطيئة، لكنها موثوقة وجودتها مضمونة. ويقول الكاتب إن هذه الحالة ليست معزولة، بل يعبر كثير من مهندسي القطاع عن رأي مشابه.

فأين تكمن مشكلة البرمجة بالذكاء الاصطناعي؟ كتب أحد المبرمجين في مدونته^[4]، متحدّثًا عن تجربته في فحص شيفرة الذكاء الاصطناعي على مدى نصف عام. وهو يرى أن وكيل الذكاء الاصطناعي أشبه بموظف جديد بارع جدًّا؛ تبدو الشيفرة التي يكتبها جيدة، لكنه لافتقاره إلى المعرفة الخلفية بالمجال لا يراعي دائمًا مختلف القيود والشروط الحديّة والحالات الاستثنائية، كما يسهل وقوعه في مشكلات أمنية. وفي الواقع، صدرت مؤخرًا ورقة بحثية عن «تآكل القيود: هشاشة وكلاء LLM في توليد شيفرة الواجهة الخلفية»^[5]، وقد تطرقت هذه الورقة إلى المشكلة ذاتها:

كشفت دراستنا ظاهرة «تآكل القيود»: فمع ازدياد المتطلبات البنيوية يتراجع أداء الوكلاء تراجعًا واضحًا. فمن المهام المرجعية إلى المهام المحددة تحديدًا كاملاً، انخفض معدل اجتياز التأكيدات في التكوينات الجيدة الأداء بمقدار 30 نقطة مئوية في المتوسط، وهبط في بعض التكوينات الأضعف أداءً إلى مستوى

يقارب الصفر. وقد كشف تحليل الحساسية للإطار اختلافات كبيرة في الأداء: فالوكلاء يبلون بلاءً حسنًا في الأطر البسيطة والصريحة (كـ Flask)، لكن أداءهم المتوسط ينهار في البيئات المعتمدة اعتمادًا شديدًا على الأعراف (كـ Django و FastAPI). وأخيرًا، يُظهر تحليل الأخطاء أن عيوب طبقة البيانات (مثل بناء الاستعلامات الخاطئ والاستثناءات وقت التشغيل في ORM) هي المصدر الرئيسي لهذه المشكلات. وتؤكد هذه الدراسة أن تلبية المتطلبات الوظيفية والبنوية في وقت واحد لا تزال تحديًا محوريًا لم يُحل بعد أمام وكلاء البرمجة.

وقد أشار مدير تقني (CTO) في شركة هندية في مقال حديث^[6] إلى الآتي. وقد ذكر أن الشيفرة التي يكتبها الذكاء الاصطناعي عرضة للمشكلات عند التشغيل الفعلي في بيئة الإنتاج، والسبب أنه «في الأنظمة الكبيرة، يجب أن تتوافق الشيفرة مع أنماط البنية وسلاسل التبعيات وقواعد العمل ومتطلبات الامتثال وضوابط الأمن وعمليات الإصدار؛ وإذا تجاهلت الشيفرة المولدة بالذكاء الاصطناعي هذه القيود، انتقل تركيز العمل من التطوير إلى الإصلاح والتقويم اللاحقين، ما يؤدي إلى إطالة دورة مراجعة الشيفرة، ونقص تغطية الاختبارات، وظهور مخاطر أمنية، وفشل عمليات الدمج، وعيوب في بيئة الإنتاج». ومن جرب البرمجة بالذكاء الاصطناعي فعليًا يدرك أن هذه المشكلات واسعة الانتشار حقًا.

فالشيفرة التي يكتبها الذكاء الاصطناعي سريعة، لكن بسبب هذه المشكلات المذكورة أعلاه يجد المطورون أنفسهم مضطرين إلى تفحص الشيفرة المولدة بالذكاء الاصطناعي بعناية قبل دمجها في مستودع الشيفرة. وهذا يطرح مسألتين. الأولى أن سرعة تفحص الشيفرة أدنى بكثير من سرعة توليدها، ما يجعل زيادة الكفاءة التي يوفرها الذكاء الاصطناعي تنقل تقلصًا كبيرًا. والثانية أن المبرمجين الجدد يفتقدون الفرصة لإمساك الشيفرة وكتابتها بأيديهم، فلا تنمو قدراتهم؛ وحين يُسرح الجيل القديم من المبرمجين أو يُحال إلى التقاعد، ستتباطأ سرعة تفحص الشيفرة أكثر، بل لن يبقى التفحص قادرًا على كشف المشكلات.

بل إن بعض الشركات تُعَلِّق آملًا كبيرة على البرمجة بالذكاء الاصطناعي، فتضغط فترات التطوير ضغطًا حادًا بحيث لا يجد المبرمجون وقتًا لفحص الشيفرة بجدية. ومن وجهة نظر المطورين، الشركة ليست ملكهم؛ فإن طلب الرئيس الأعلى السرعة، اكتفوا بفحص سطحي أو دمجها دون فحص أصلاً. وإن ظهرت مشكلة بعد التشغيل، فالأسوأ أن يستقيلوا. وسواء كان الأمر إجهادًا في التفحص أو تفحصًا غير كافٍ، فالنتيجة واحدة: تختبئ في شيفرة بيئة الإنتاج أكوام من الأخطاء والروائح الكريهة في الشيفرة وما شابه، بينما من يتولى صيانة هذه البرمجيات لا يعرف الشيفرة في الحقيقة معرفة جيدة. وكل هذا الدين التقني ينتج ما يسمى «فجوة قابلية الصيانة» (The Maintainability Gap)^[7].

تواصل البرمجة بمساعدة الذكاء الاصطناعي تقدمها السريع، وبلغ حجم الشيفرة التي ينتجها أكثر المطورين حماسًا مستوى قياسيًا غير مسبوق. غير أن تحليل بيانات تغييرات الشيفرة على مدى أربع سنوات يُظهر أن مؤشرات قابلية الصيانة في تراجع: انخفضت الاستدعاءات بين الملفات (وهي تجلّ لإعادة استخدام الشيفرة) بنسبة 35%؛ وتراجعت عمليات نقل أسطر الشيفرة في إعادة الهيكلة بنسبة 70% مقارنة بمستوى 2022؛ كما تراجع العمل على صيانة الشيفرة القديمة العالقة بنسبة 74%. وفي الوقت نفسه، نلاحظ تصاعدًا في ظواهر مقلقة: زاد سلوك «النسخ/اللصق» داخل الإيداع الواحد بنسبة 41%، وتكررت كتل الشيفرة بنسبة 81%، وزادت البنى البرمجية التي تحجب الأخطاء بنسبة 47%، وارتفع معدل فقدان الشيفرة (churn) في غضون أسبوعين بنسبة 15%. فرغم أن إنتاج الشيفرة ارتفع فعليًا

ارتفاعًا كبيرًا، فإن الدين التقني المتراكم معه لا يقل خطرًا، وهو يتركز بالأساس عند المطورين الذين لم يدركوا بعد أن بعض أنماط كتابة الشيفرة تهدد قابلية الصيانة طويلة الأمد للمستودع.

أما عن الدين التقني، فتقول الورقة «[8] Debt Behind the AI Boom» من خلال دراسة واسعة النطاق إن «رائحة الشيفرة هي أكثر أنواع المشكلات شيوعًا، إذ تبلغ نسبتها 89.3%»، و«من بين الإيداعات التي أنتجتها مختلف مساعدات البرمجة بالذكاء الاصطناعي، أدخل أكثر من 15% منها مشكلة واحدة على الأقل. والأهم من ذلك أن 22.7% من المشكلات التي أدخلها الذكاء الاصطناعي لا تزال قائمة في أحدث إصدار من مستودع الشيفرة». و«تدل هذه النتائج على أن الشيفرة المولدة بالذكاء الاصطناعي قد تحمّل مشروعات البرمجيات الفعلية تكاليف صيانة طويلة الأمد، كما تُبرز ضرورة تعزيز ضمان الجودة في التطوير بمساعدة الذكاء الاصطناعي».

Figure 3 

المصدر: «Debt Behind the AI Boom»

وأما ورقة «[9] Faster Code, Deeper Debt?» فتقول إن «نماذج LLM كثيرًا ما تزيد من حدة الأشكال التقليدية للدين التقني (ولا سيما الدين في الشيفرة والتصميم والتوثيق)، وتستحدث في الوقت نفسه أنواعًا جديدة من الدين خاصة بها. ومن الجدير بالملاحظة أننا تعرفنا على نوع من "دين الدمج السريع"، أي التضحية بالجودة طلبًا للسرعة، فتؤدي الشيفرة المولدة بسرعة إلى سلسلة من ردود الفعل، تولّد دينًا في الحكامة وتزيد تكاليف الصيانة على المدى الطويل. ومن الفئات الناشئة الأخرى للدين دين التوجيهات (prompt)، والدين الأخلاقي، والدين البياني، ودين التتبع، وهي تعكس التحديات الفريدة التي تطرحها تطبيقات LLM».

لا تؤدي البرمجة بالذكاء الاصطناعي إلى تراكم الدين التقني فحسب، بل تجلب ديونًا أخرى أيضًا. فقد أشارت عالمة الحاسوب Margaret-Anne Storey في ورقتها «From Technical Debt to Cognitive and Intent Debt» [10] إلى أمرين: وفيما يتجاوز الدين التقني، ومع تجاوز سرعة توليد الذكاء الاصطناعي للشيفرة سرعة فهم الفريق لها، يتراكم شكلان من «الدين» كثيرًا ما يُستهان بهما، أولهما «الدين المعرفي» (cognitive debt)، أي ضياع التوافق داخل الفريق؛ وثانيهما «دين النوايا» (intent debt)، أي غياب الأساس المنطقي المُخَرَج إلى الخارج الذي يحتاجه المطورون ووكلاء الذكاء الاصطناعي للتعامل الآمن مع الشيفرة. وبكلام أوضح: قد يؤدي الاستخدام الشامل للبرمجة بالذكاء الاصطناعي إلى ألا يعرف أحد في الفريق كله ما الذي يفعله. وتركيز القطاع اليوم لا يزال على الدين التقني، أما هذه المسائل الأعمق فلم تنكشف بعد انكشافًا كاملاً.

وبعد كل هذا، أظن أن القارئ الكريم قد بلغ الجواب في نفسه. لقد كررت في إجابات كثيرة (مثل: هل تعتقد أن البرمجة بالذكاء الاصطناعي ستحل حقًا محل المبرمجين في المستقبل؟ وكيف ننظر إلى توقع مسؤول الذكاء الاصطناعي في مايكروسوفت بأن «معظم وظائف الياقات البيضاء سيستبدلها الذكاء الاصطناعي استبدالًا كاملاً خلال 18 شهرًا»؟) رأيًا واحدًا: سيغيّر الذكاء الاصطناعي طريقة عمل ذوي الياقات البيضاء تغييرًا عميقًا، لكنه لا يستطيع أن يُقصي جميع المبرمجين، وأحرى أن يُقيل جميع ذوي الياقات البيضاء. وقد أثبتت الوقائع الفعلية صحة رأيي.

ومع تبدد أسطورة أن الذكاء الاصطناعي سيحل محل كل ذوي الياقات البيضاء، بدأت تظهر في سوق رأس المال مؤشرات على انفجار الفقاعة. وقد استنتجت سابقًا، على افتراض رفع كفاءة تطوير البرمجيات بنسبة 30%، أن فقاعة الذكاء الاصطناعي موجودة فعليًا وأن حجمها أكبر من فقاعة الإنترنت (انظر: «ما حجم فقاعة الذكاء الاصطناعي؟ وهل ستتجاوز أزمته أزمة 2008 المالية؟»). وأما حديث مدراء شركات النماذج الكبيرة عن «دمار الذكاء الاصطناعي للبشرية» في الآونة الأخيرة، فهو ليس إلا جنون اللحظة الأخيرة قبل الفناء.

المراجع

- The AI revolution in software development <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>
- Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools. https://download.ssrn.com/2026/9/6/6843118.pdf?response-content-disposition=inline&X-Amz-Security-BGEXoeayHvn1JN4NCfFjioMOP%2B8czxCNN1WRk1xigrdJae4Ew52iInuVz%2FDmizuSzgvtww9A%3D%3D&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Date=20260919T025315Z&X-Amz-SignedHeaders=host&X-Amz-Expires=300&X-Amz-Credential=ASIAUPUUPRWEYZTNYLVN%2F20260919%2Fus-east-1%2Fs3%2Faws4_request&X-Amz-e=4ba6ead523ab5e286d4d945a3af8b9fbf80ada4dle8f200c179fed48d0732b3&abstractId=6843118
- On AI Coding and Its Discontents <https://calnewport.com/on-ai-coding-and-its-discontents>
- I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25
- Constraint Decay: The Fragility of LLM Agents in Backend Code Generation <https://arxiv.org/pdf/2605.06445>
- The Productivity Illusion at the Heart of Enterprise AI Coding <https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding>
- The Maintainability Gap: AI Code Quality in 2026 https://www.gitclear.com/the_ai_code_quality_maintainability_gap

Constraint Decay: The Fragility of LLM Agents in Backend Code Generation [5]

中文原文: Constraint Decay: The Fragility of LLM Agents in Backend Code Generation
<https://arxiv.org/pdf/2605.06445>

The Productivity Illusion at the Heart of Enterprise AI Coding [6]

中文原文: The Productivity Illusion at the Heart of Enterprise AI Coding
[/https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding](https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding)

The Maintainability Gap: AI Code Quality in 2026 [7]

中文原文: The Maintainability Gap: AI Code Quality in 2026
https://www.gitclear.com/the_ai_code_quality_maintainability_gap

Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild [8]

中文原文: Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild
<https://arxiv.org/abs/2603.28592>

Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs [9]

in LLM-Assisted Software Development

中文原文: Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development
<https://arxiv.org/abs/2606.14796>

From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI [10]

中文原文: From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI
<https://arxiv.org/abs/2603.22106>

<https://zhuanlan.zhihu.com/p/2084561490930045240>: نُشر هذا المقال أصلاً على موقع تشيهو (Zhihu) بالصينية:

<https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>: المؤلف: 最最遥远的路 • صفحة تشيهو:

本文原发表于知乎专栏/回答 (中文): <https://zhuanlan.zhihu.com/p/2084561490930045240>; 作者: 最最遥远的路: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>